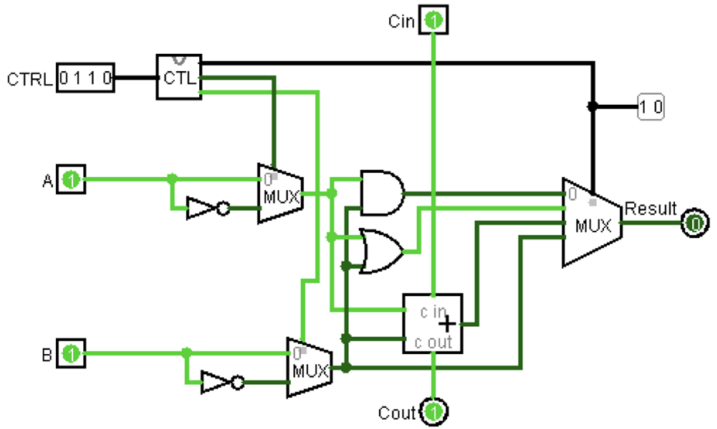
Clarification about Lab05 Questions

Q3
1 Point

Suppose a new LEGv8 instruction passed an ALUctl value of 0b1000 in to the simplified ALU we studied in class, and this ALUctl value has the effect of passing a 1 select value into the MUX fed by B and the inverted value of B, and passing a 11 value into the MUX that feeds the result. What would be the best choice of a mnemonic for this instruction?

Simplified ALU – One Bit



ALU Control: 0000=AND; 0001=OR; 0010=Add; 0110=Sub; 0111=Pass B

Clarification about Lab05 Questions

Q6
1 Point

Suppose a single cycle LEGv8 Processor had a frequency of 0.5 GHz, and you executed a program with 5,000,000 instructions. The mix of instructions is as follows:

What is the total CPU time required to run the program?

- ○ < 0.001 Seconds
- ○ < 0.01 but >= 0.001 Seconds
- ◉ < 0.1 but >= 0.01 Seconds
- ○ < 1.0 but >= 0.01 Seconds
- ○ >= 1.0 Seconds

- Both are correct answers
- The CPU time = 0.01 seconds

Project 2: FOOTv8 Pipeline Simulation

10/26/2026

Project 2: Overview

- Build a simulator for pipelined processor
- This processor processes simplified version of LEGv8 ISA (FOOT v8)

Project Rules

- Cooperation is NOT allowed.
- Project must be written in C.
- We will grade your project on remote.cs.binghamton.edu machines, Testing on any of these machines works:
  - Remote server
  - EB-G07
  - EB-Q22
  - EB-N01

Program Specifications

Stick with the output format

- Input: file containing initial memory values
- Output: simulation report

Input

```
00000000 11000fa0
00000004 0b010000
00000008 07010109
0000000c 09010105
00000010 03010120
00000014 0d010004
00000018 0e000000
```



Output

```
=== Simulation Reports ===
Non-Zero Registers....
  X0 = 4000
  X1 = 32
Instruction Count Report...
   OPCODE - CNT
     ADDI -   1
     MULI -   1
     DIVI -   1
       LD -   1
       ST -   1
     EXIT -   1
      MOV -   1

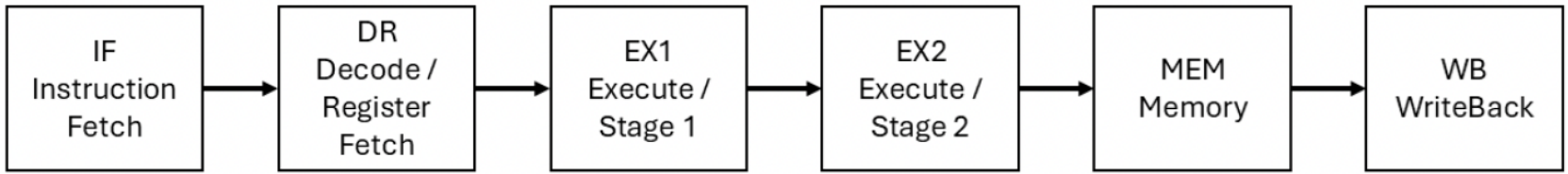
Total Instructions=7 Cycles=32 IPC= 0.22
```



Arrange instruction based on opcode value

Program Specifications

- Your simulator simulates the execution of FOOTv8 instructions.
- The first instruction is at memory address 0x0
- Simulate from the first instruction (cycle 0) till EXIT instruction reaching write-back stage
- Your simulator executes the following stages:



Program Specifications: Memory

- Instruction and data memory are combined
- Its size is 2048 four byte words
- Address range: 0x0000000 - 0x00002000
- Any word that does not appear in the input file must be initialized to zero.
- The architectural registers: X0-X31: initialized to zero
- This simulator does not support data forwarding or branch prediction

Program Specifications: Hazards

- Data hazard is detected in the DR stage.
- When a data hazard is detected, the DR and IF stages halt until hazard is resolved.
- A data hazard is resolved when pending register update get to WB stage

- Control hazard is detected in the EX2 stage and only when a branch is taken
- Instructions in IF, DR and EX1 must be squashed
- PC is updated with correct branch target
- Resume processing

Project Specifications

- Keep track of registers values
- Keep track of number of cycles and instructions that are simulated completely
- Don’t count NOP instruction in the total number of instructions

Registers Simulation: Proposed Approach

- You can simulate register as data structure with two integer values
  - Current value and future value
- Current value: value that will be read
- Future value: value that will be written the next time there is a rising edge on the clock
- The register should have at least 3 methods:
  - setRegisterValue(intval)
  - getRegisterValue()
  - pulseClock() → move new value to old value

- You can use the register component for: PC, registers (X0-X31), inter-stage data

- Remember to pulse the clock for all the registers at the same time

To help debugging your simulator

```
== cycle |    fetch |   decode |  exec1   |  exec2   |   memory | writeBack|
== ------+----------+----------+----------+----------+----------+----------|
==     0 |        0 | ffffffff | ffffffff | ffffffff | ffffffff | ffffffff |
==     1 |        4 |        0 | ffffffff | ffffffff | ffffffff | ffffffff |
==     2 |        8 |        4 |        0 | ffffffff | ffffffff | ffffffff |
Data hazard... register X0 will be written by the instruction in the ex1 stage
==     3 |        8 |        4 |        4 |        0 | ffffffff | ffffffff |
Data hazard... register X0 will be written by the instruction in the ex2 stage
==     4 |        8 |        4 |        4 |        4 |        0 | ffffffff |
Data hazard... register X0 will be written by the instruction in the mem stage
==     5 |        8 |        4 |        4 |        4 |        4 |        0 |
Data hazard... register X0 will be written by the instruction in the wb stage
==     6 |        8 |        4 |        4 |        4 |        4 |        4 |
==     7 |        c |        8 |        4 |        4 |        4 |        4 |
Data hazard... register X1 will be written by the instruction in the ex1 stage
==     8 |        c |        8 |        8 |        4 |        4 |        4 |
Data hazard... register X1 will be written by the instruction in the ex2 stage
==     9 |        c |        8 |        8 |        8 |        4 |        4 |
Data hazard... register X1 will be written by the instruction in the mem stage
==    10 |        c |        8 |        8 |        8 |        8 |        4 |
Data hazard... register X1 will be written by the instruction in the wb stage
==    11 |        c |        8 |        8 |        8 |        8 |        8 |
==    12 |       10 |        c |        8 |        8 |        8 |        8 |
Data hazard... register X1 will be written by the instruction in the ex1 stage
==    13 |       10 |        c |        c |        8 |        8 |        8 |
```

Notes

- Stick with project directory hierarchy and names
- Stick with output format

Deadline
Nov 18th @ 11PM